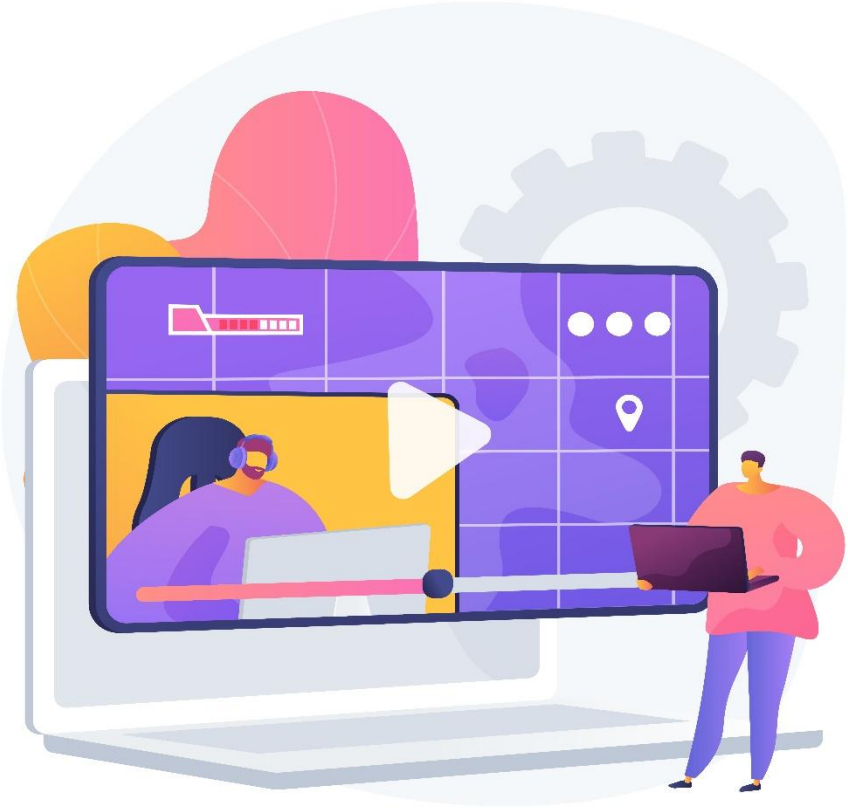
VIDEO STREAMING MANAGEMENT SYSTEM



- JAY RAVINDRA GAJARE

VIDEO STREMING MANAGEMENT SYSTEM

A Video Streaming Management System is a project that simplifies and automates the management of digital streaming platforms. This system ensures efficient handling of core OTT operations such as managing users, subscription plans, video content, genres, watch history, and payment transactions.

This project demonstrates the implementation of a Video Streaming Database Management System using MySQL. It includes designing and managing relational tables, inserting real-world data, establishing relationships using primary and foreign keys, and executing advanced SQL queries such as JOINs, subqueries, and aggregate functions. The main objective of this project is to showcase strong skills in database design, data manipulation, and analytical querying while simulating a real-world OTT platform environment.

PROJECT AIM

- **Video Content Management:**
  Add, update, and remove video content from the streaming platform. Track video details such as title, genre, duration, release year, language, rating, and availability status for movies and web series.
- **User Management:**
  Maintain a complete record of platform users, including user names, email addresses, contact details, registration dates, subscription status, and viewing activity.
- **Subscription Management:**
  Manage different subscription plans, including plan names, pricing, duration, and benefits. Track which users are subscribed to which plans and monitor subscription validity.
- **Watch History Tracking:**
  Track the watch history of users, including videos watched, watch dates, and watch duration. Monitor user engagement and content popularity to support analytical reporting.

- **Payment Management:**
Maintain payment records related to subscriptions, including payment amount, payment date, payment method, and transaction status, ensuring accurate revenue tracking.
- **Device Management:**
Manage information about devices used by users to access the platform, including device type, login activity, and supported limits per subscription plan.

OBJECTIVES

1. **Set up the Video Streaming Database System:**
   Create and populate the database with relational tables for users, subscription plans, user subscriptions, videos, genres, watch history, payments, and devices to accurately represent a real-world OTT platform.
2. **Perform CRUD Operations:**
   Implement Create, Read, Update, and Delete (CRUD) operations to efficiently manage user accounts, video content, subscriptions, and payment records within the database.
3. **Execute Advanced SQL Queries:**
   Develop and execute advanced SQL queries, including JOINs, subqueries, and aggregate functions, to analyze user activity, content popularity, and revenue generation.

ER Diagram VIDEO STREAMING MANAGEMENT SYSTEM

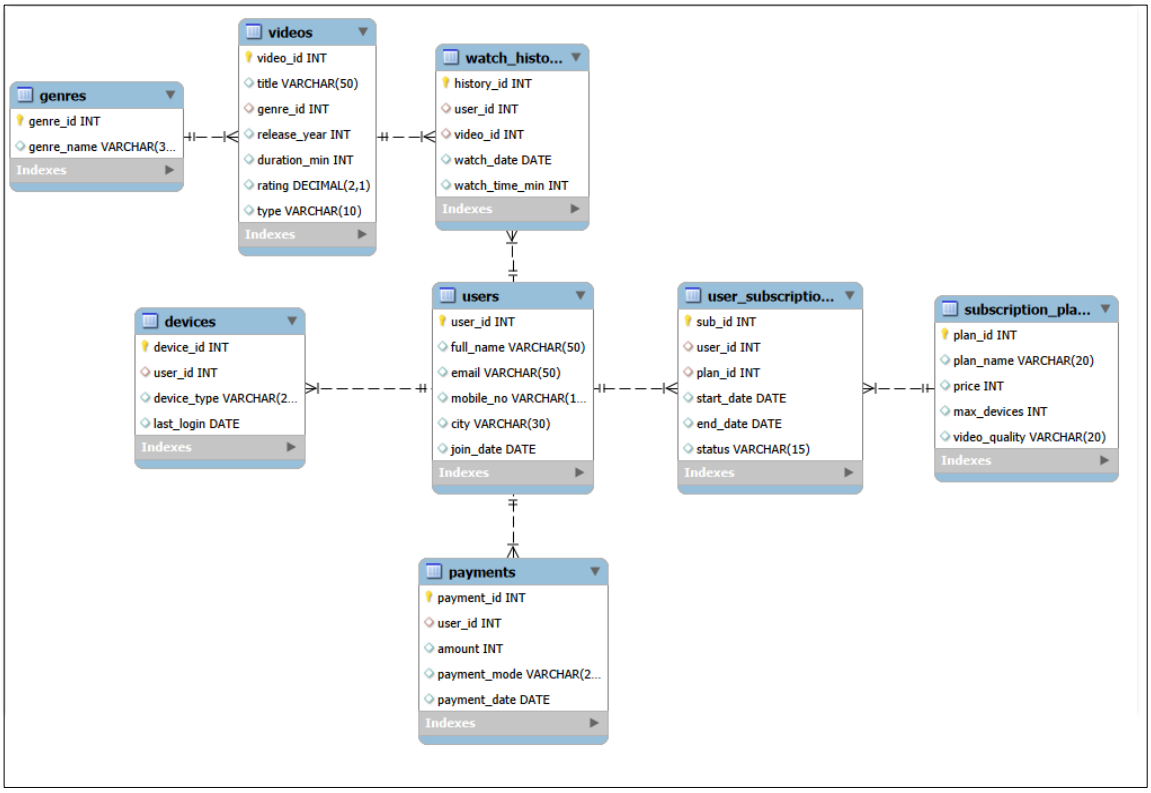


Table Description:

1) **users**

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	
full_name	varchar(50)	YES		NULL	
email	varchar(50)	YES		NULL	
mobile_no	varchar(15)	YES		NULL	
city	varchar(30)	YES		NULL	
join_date	date	YES		NULL	

2) **Subscription_plans**

Result Grid | Filter Rows: | Export: | Wra

Field	Type	Null	Key	Default	Extra
plan_id	int	NO	PRI	NULL	
plan_name	varchar(20)	YES		NULL	
price	int	YES		NULL	
max_devices	int	YES		NULL	
video_quality	varchar(20)	YES		NULL	

3) **user_subscription**

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
sub_id	int	NO	PRI	NULL	
user_id	int	YES	MUL	NULL	
plan_id	int	YES	MUL	NULL	
start_date	date	YES		NULL	
end_date	date	YES		NULL	
status	varchar(15)	YES		NULL	

4) **genres**

Result Grid | Filter Rows: | Export: | V

Field	Type	Null	Key	Default	Extra
genre_id	int	NO	PRI	NULL	
genre_name	varchar(30)	YES		NULL	

5) videos

Result Grid | Filter Rows: | Export: | Wr

Field	Type	Null	Key	Default	Extra
video_id	int	NO	PRI	NULL	
title	varchar(50)	YES		NULL	
genre_id	int	YES	MUL	NULL	
release_year	int	YES		NULL	
duration_min	int	YES		NULL	
rating	decimal(2, 1)	YES		NULL	
type	varchar(10)	YES		NULL	

6) watch_history

Result Grid | Filter Rows: | Export:

Field	Type	Null	Key	Default	Extra
history_id	int	NO	PRI	NULL	
user_id	int	YES	MUL	NULL	
video_id	int	YES	MUL	NULL	
watch_date	date	YES		NULL	
watch_time_min	int	YES		NULL	

7) devices

Result Grid | Filter Rows: | Export: | W

Field	Type	Null	Key	Default	Extra
device_id	int	NO	PRI	NULL	
user_id	int	YES	MUL	NULL	
device_type	varchar(20)	YES		NULL	
last_login	date	YES		NULL	

8) payments

Result Grid | Filter Rows: | Export: | Wrap

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	
user_id	int	YES	MUL	NULL	
amount	int	YES		NULL	
payment_mode	varchar(20)	YES		NULL	
payment_date	date	YES		NULL	

CREATING DATABASE:

CREATE DATABASE library_management_system;
use library_management_system;

Table Creation & Insertion Commands:

1) Create Table users

```
CREATE TABLE users (
user_id INT PRIMARY KEY,
full_name VARCHAR(50),
email VARCHAR(50),
mobile_no VARCHAR(15),
city VARCHAR(30),
join_date DATE);
```

Inserting Values into users**:**

```
INSERT INTO users VALUES
(1,'Rahul Sharma','rahul@gmail.com','9876543210','Mumbai','2024-01-05'),
(2,'Amit Patil','amit@gmail.com','9876543211','Pune','2024-01-08'),
(3,'Sneha Joshi','sneha@gmail.com','9876543212','Nashik','2024-01-10'),
(4,'Rohit Deshmukh','rohit@gmail.com','9876543213','Nagpur','2024-01-12'),
(5,'Pooja Kulkarni','pooja@gmail.com','9876543214','Kolhapur','2024-01-15'),
(6,'Akash Verma','akash@gmail.com','9876543215','Delhi','2024-01-18'),
(7,'Neha Singh','neha@gmail.com','9876543216','Noida','2024-01-20'),
(8,'Kunal Mehta','kunal@gmail.com','9876543217','Ahmedabad','2024-01-22'),
(9,'Riya Shah','riya@gmail.com','9876543218','Surat','2024-01-25'),
(10,'Sahil Khan','sahil@gmail.com','9876543219','Bhopal','2024-01-27'),
(11,'Anjali Gupta','anjali@gmail.com','9876543220','Indore','2024-02-01'),
(12,'Vikas Yadav','vikas@gmail.com','9876543221','Lucknow','2024-02-03'),
(13,'Manish Jain','manish@gmail.com','9876543222','Jaipur','2024-02-05'),
(14,'Priya Malhotra','priya@gmail.com','9876543223','Chandigarh','2024-02-07'),
(15,'Deepak Rao','deepak@gmail.com','9876543224','Hyderabad','2024-02-10'),
(16,'Swati Iyer','swati@gmail.com','9876543225','Chennai','2024-02-12'),
(17,'Arjun Nair','arjun@gmail.com','9876543226','Kochi','2024-02-14'),
(18,'Nitin Pawar','nitin@gmail.com','9876543227','Aurangabad','2024-02-16'),
(19,'Kavita More','kavita@gmail.com','9876543228','Satara','2024-02-18'),
(20,'Yash Thakur','yash@gmail.com','9876543229','Raipur','2024-02-20');
```

SELECT * FROM users;

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import:

user_id	full_name	email	mobile_no	city	join_date
1	Rahul Sharma	rahul@gmail.com	9876543210	Mumbai	2024-01-05
2	Amit Patil	amit@gmail.com	9876543211	Pune	2024-01-08
3	Sneha Joshi	sneha@gmail.com	9876543212	Nashik	2024-01-10
4	Rohit Deshmukh	rohit@gmail.com	9876543213	Nagpur	2024-01-12
5	Pooja Kulkarni	pooja@gmail.com	9876543214	Kolhapur	2024-01-15
6	Akash Verma	akash@gmail.com	9876543215	Delhi	2024-01-18
7	Neha Singh	neha@gmail.com	9876543216	Noida	2024-01-20
8	Kunal Mehta	kunal@gmail.com	9876543217	Ahmedabad	2024-01-22
9	Riya Shah	riya@gmail.com	9876543218	Surat	2024-01-25
10	Sahil Khan	sahil@gmail.com	9876543219	Bhopal	2024-01-27
11	Anjali Gupta	anjali@gmail.com	9876543220	Indore	2024-02-01
12	Vikas Yadav	vikas@gmail.com	9876543221	Lucknow	2024-02-03
13	Manish Jain	manish@gmail.com	9876543222	Jaipur	2024-02-05
14	Priya Malhotra	priya@gmail.com	9876543223	Chandigarh	2024-02-07
15	Deepak Rao	deepak@gmail.com	9876543224	Hyderabad	2024-02-10
16	Swati Iyer	swati@gmail.com	9876543225	Chennai	2024-02-12
17	Arjun Nair	arjun@gmail.com	9876543226	Kochi	2024-02-14
18	Nitin Pawar	nitin@gmail.com	9876543227	Aurangabad	2024-02-16
19	Kavita More	kavita@gmail.com	9876543228	Satara	2024-02-18
20	Yash Thakur	yash@gmail.com	9876543229	Raipur	2024-02-20
NULL	NULL	NULL	NULL	NULL	NULL

2) **Create Table subscription_plans**

```
CREATE TABLE  subscription_plans (
plan_id INT PRIMARY KEY,
 plan_name VARCHAR(20),
 price INT,
 max_devices INT,
 video_quality VARCHAR(20));
```

Inserting Values into subscription_plans:

```
INSERT INTO subscription_plans  VALUES
(1,'Mobile',149,1,'SD'),
(2,'Basic',199,1,'HD'),
(3,'Standard',499,2,'Full HD'),
(4,'Premium',799,4,'4K');
```

```
SELECT * FROM subscription_plans;
```

OUTPUT:

Result Grid | Filter Rows: | Edit:

plan_id	plan_name	price	max_devices	video_quality
1	Mobile	149	1	SD
2	Basic	199	1	HD
3	Standard	499	2	Full HD
4	Premium	799	4	4K
NULL	NULL	NULL	NULL	NULL

3) **Create Table user_subscriptions**

```
CREATE TABLE user_subscriptions (
sub_id INT PRIMARY KEY,
user_id INT,
plan_id INT,
start_date DATE,
end_date DATE,
status VARCHAR(15),
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (plan_id) REFERENCES subscription_plans(plan_id));
```

Inserting Values into user_subscriptions:

```
INSERT INTO user_subscriptions VALUES
(1,1,3,'2024-01-05','2024-02-05','Active'),
(2,2,2,'2024-01-08','2024-02-08','Active'),
(3,3,4,'2024-01-10','2024-02-10','Expired'),
(4,4,1,'2024-01-12','2024-02-12','Active'),
(5,5,3,'2024-01-15','2024-02-15','Active'),
(6,6,4,'2024-01-18','2024-02-18','Expired'),
(7,7,2,'2024-01-20','2024-02-20','Active'),
(8,8,3,'2024-01-22','2024-02-22','Active'),
(9,9,1,'2024-01-25','2024-02-25','Expired'),
(10,10,4,'2024-01-27','2024-02-27','Active'),
(11,11,2,'2024-02-01','2024-03-01','Active'),
(12,12,3,'2024-02-03','2024-03-03','Active'),
(13,13,1,'2024-02-05','2024-03-05','Expired'),
(14,14,4,'2024-02-07','2024-03-07','Active'),
(15,15,3,'2024-02-10','2024-03-10','Active'),
(16,16,2,'2024-02-12','2024-03-12','Active'),
(17,17,4,'2024-02-14','2024-03-14','Expired'),
(18,18,1,'2024-02-16','2024-03-16','Active'),
(19,19,2,'2024-02-18','2024-03-18','Active'),
(20,20,3,'2024-02-20','2024-03-20','Active');
```

SELECT * FROM <u>user_subscriptions</u>;

<u>OUTPUT:</u>

Result Grid | Filter Rows: | Edit:

sub_id	user_id	plan_id	start_date	end_date	status
1	1	3	2024-01-05	2024-02-05	Active
2	2	2	2024-01-08	2024-02-08	Active
3	3	4	2024-01-10	2024-02-10	Expired
4	4	1	2024-01-12	2024-02-12	Active
5	5	3	2024-01-15	2024-02-15	Active
6	6	4	2024-01-18	2024-02-18	Expired
7	7	2	2024-01-20	2024-02-20	Active
8	8	3	2024-01-22	2024-02-22	Active
9	9	1	2024-01-25	2024-02-25	Expired
10	10	4	2024-01-27	2024-02-27	Active
11	11	2	2024-02-01	2024-03-01	Active
12	12	3	2024-02-03	2024-03-03	Active
13	13	1	2024-02-05	2024-03-05	Expired
14	14	4	2024-02-07	2024-03-07	Active
15	15	3	2024-02-10	2024-03-10	Active
16	16	2	2024-02-12	2024-03-12	Active
17	17	4	2024-02-14	2024-03-14	Expired
18	18	1	2024-02-16	2024-03-16	Active
19	19	2	2024-02-18	2024-03-18	Active
20	20	3	2024-02-20	2024-03-20	Active
NULL	NULL	NULL	NULL	NULL	NULL

4) **Create Table genres**

CREATE TABLE genres
(genre_id INT PRIMARY KEY,
genre_name VARCHAR(30));

Inserting Values into genres:

INSERT INTO genres
VALUES
(1,'Action'),
(2,'Drama'),
(3,'Comedy'),
(4,'Romance'),
(5,'Thriller');

SELECT * FROM genres;

OUTPUT :

Result Grid | Filter Rows:

genre_id	genre_name
1	Action
2	Drama
3	Comedy
4	Romance
5	Thriller
NULL	NULL

Drama

5) **Create Table videos**

```
CREATE TABLE videos (
video_id INT PRIMARY KEY,
title VARCHAR(50),
genre_id INT,
release_year INT,
duration_min INT,
rating DECIMAL(2,1),
type VARCHAR(10),
FOREIGN KEY (genre_id) REFERENCES genres(genre_id)
);
```

Inserting Values into videos:

```
INSERT INTO videos
VALUES
(1,'War Zone',1,2023,140,8.4,'Movie'),
(2,'Family Man',2,2022,45,8.7,'Series'),
(3,'Laugh Club',3,2024,30,8.0,'Series'),
(4,'Love Forever',4,2021,120,7.6,'Movie'),
(5,'Crime Files',5,2023,50,8.2,'Series'),
(6,'Mission X',1,2022,150,8.1,'Movie'),
(7,'City Life',2,2020,110,7.4,'Movie'),
(8,'Standup Nights',3,2021,35,7.9,'Series'),
(9,'Heart Beats',4,2022,130,7.8,'Movie'),
(10,'Dark Truth',5,2024,55,8.6,'Series'),
(11,'Fighter Squad',1,2023,145,8.3,'Movie'),
(12,'Broken Dreams',2,2021,125,7.5,'Movie'),
(13,'Comedy Adda',3,2023,40,8.1,'Series'),
(14,'Forever Us',4,2024,115,7.7,'Movie'),
(15,'The Chase',5,2022,135,8.0,'Movie'),
(16,'Action Heroes',1,2024,150,8.5,'Movie'),
(17,'Life Stories',2,2023,60,7.9,'Series'),
(18,'Fun Factory',3,2022,25,7.6,'Series'),
(19,'Love Again',4,2023,120,7.8,'Movie'),
(20,'Hidden Secrets',5,2024,50,8.4,'Series');
```

SELECT * FROM videos;

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import:

video_id	title	genre_id	release_year	duration_min	rating	type
1	War Zone	1	2023	140	8.4	Movie
2	Family Man	2	2022	45	8.7	Series
3	Laugh Club	3	2024	30	8.0	Series
4	Love Forever	4	2021	120	7.6	Movie
5	Crime Files	5	2023	50	8.2	Series
6	Mission X	1	2022	150	8.1	Movie
7	City Life	2	2020	110	7.4	Movie
8	Standup Nights	3	2021	35	7.9	Series
9	Heart Beats	4	2022	130	7.8	Movie
10	Dark Truth	5	2024	55	8.6	Series
11	Fighter Squad	1	2023	145	8.3	Movie
12	Broken Dreams	2	2021	125	7.5	Movie
13	Comedy Adda	3	2023	40	8.1	Series
14	Forever Us	4	2024	115	7.7	Movie
15	The Chase	5	2022	135	8.0	Movie
16	Action Heroes	1	2024	150	8.5	Movie
17	Life Stories	2	2023	60	7.9	Series
18	Fun Factory	3	2022	25	7.6	Series
19	Love Again	4	2023	120	7.8	Movie
20	Hidden Secrets	5	2024	50	8.4	Series
NULL	NULL	NULL	NULL	NULL	NULL	NULL

6) **Create Table watch_history**

```
CREATE TABLE watch_history (

history_id INT PRIMARY KEY,
user_id INT,
video_id INT,
watch_date DATE,
watch_time_min INT,
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (video_id) REFERENCES videos(video_id));
```

Inserting Values into watch_history:

```
INSERT INTO watch_history
VALUES
(1,1,2,'2024-02-01',40),
(2,2,1,'2024-02-02',120),
(3,3,3,'2024-02-03',25),
(4,4,4,'2024-02-04',90),
(5,5,5,'2024-02-05',50),
(6,6,6,'2024-02-06',130),
(7,7,7,'2024-02-07',100),
(8,8,8,'2024-02-08',30),
(9,9,9,'2024-02-09',110),
(10,10,10,'2024-02-10',45),
(11,11,11,'2024-02-11',140),
(12,12,12,'2024-02-12',120),
(13,13,13,'2024-02-13',35),
(14,14,14,'2024-02-14',100),
(15,15,15,'2024-02-15',130),
(16,16,16,'2024-02-16',145),
(17,17,17,'2024-02-17',55),
(18,18,18,'2024-02-18',20),
(19,19,19,'2024-02-19',115),
(20,20,20,'2024-02-20',48);
```

SELECT * FROM watch_history;

OUTPUT:

Result Grid | Filter Rows: | Edit:

history_id	user_id	video_id	watch_date	watch_time_min
1	1	2	2024-02-01	40
2	2	1	2024-02-02	120
3	3	3	2024-02-03	25
4	4	4	2024-02-04	90
5	5	5	2024-02-05	50
6	6	6	2024-02-06	130
7	7	7	2024-02-07	100
8	8	8	2024-02-08	30
9	9	9	2024-02-09	110
10	10	10	2024-02-10	45
11	11	11	2024-02-11	140
12	12	12	2024-02-12	120
13	13	13	2024-02-13	35
14	14	14	2024-02-14	100
15	15	15	2024-02-15	130
16	16	16	2024-02-16	145
17	17	17	2024-02-17	55
18	18	18	2024-02-18	20
19	19	19	2024-02-19	115
20	20	20	2024-02-20	48
NULL	NULL	NULL	NULL	NULL

7) **Create Table devices**

```
CREATE TABLE devices
(device_id INT PRIMARY KEY,
user_id INT,
device_type VARCHAR(20),
last_login DATE,
FOREIGN KEY (user_id) REFERENCES users(user_id));
```

Inserting Values into devices :

```
INSERT INTO videos
VALUES
(1,1,'Mobile','2024-02-20'),
(2,2,'Smart TV','2024-02-20'),
(3,3,'Laptop','2024-02-19'),
(4,4,'Tablet','2024-02-18'),
(5,5,'Mobile','2024-02-20'),
(6,6,'Smart TV','2024-02-17'),
(7,7,'Laptop','2024-02-16'),
(8,8,'Mobile','2024-02-15'),
(9,9,'Tablet','2024-02-14'),
(10,10,'Smart TV','2024-02-13');
```

SELECT * FROM devices;

OUTPUT:

Result Grid | Filter Rows:

device_id	user_id	device_type	last_login
1	1	Mobile	2024-02-20
2	2	Smart TV	2024-02-20
3	3	Laptop	2024-02-19
4	4	Tablet	2024-02-18
5	5	Mobile	2024-02-20
6	6	Smart TV	2024-02-17
7	7	Laptop	2024-02-16
8	8	Mobile	2024-02-15
9	9	Tablet	2024-02-14
10	10	Smart TV	2024-02-13
NULL	NULL	NULL	NULL

8) **Create Table payments**

```
CREATE TABLE payment(
payment_id INT PRIMARY KEY,
user_id INT,
amount INT,
payment_mode VARCHAR(20),
payment_date DATE,
FOREIGN KEY (user_id) REFERENCES  users(user_id));
```

Inserting Values into payments:

```
INSERT INTO payments
VALUES
(1,1,499,'UPI','2024-01-05'),
(2,2,199,'Card','2024-01-08'),
(3,3,799,'NetBanking','2024-01-10'),
(4,4,149,'UPI','2024-01-12'),
(5,5,499,'Card','2024-01-15'),
(6,6,799,'UPI','2024-01-18'),
(7,7,199,'Card','2024-01-20'),
(8,8,499,'NetBanking','2024-01-22'),
(9,9,149,'UPI','2024-01-25'),
(10,10,799,'Card','2024-01-27');
```

```
SELECT * FROM payments;
```

OUTPUT:

Result Grid | Filter Rows: | Edit:

payment_id	user_id	amount	payment_mode	payment_date
1	1	499	UPI	2024-01-05
2	2	199	Card	2024-01-08
3	3	799	NetBanking	2024-01-10
4	4	149	UPI	2024-01-12
5	5	499	Card	2024-01-15
6	6	799	UPI	2024-01-18
7	7	199	Card	2024-01-20
8	8	499	NetBanking	2024-01-22
9	9	149	UPI	2024-01-25
10	10	799	Card	2024-01-27
NULL	NULL	NULL	NULL	NULL

BASIC QUESTIONS

1. Users who joined after 1 Jan 2024

```
SELECT full_name, city, join_date
FROM users
WHERE join_date > '2024-01-01';
```

OUTPUT:

Result Grid | Filter Rows:

full_name	city	join_date
Rahul Sharma	Mumbai	2024-01-05
Amit Patil	Pune	2024-01-08
Sneha Joshi	Nashik	2024-01-10
Rohit Deshmukh	Nagpur	2024-01-12
Pooja Kulkarni	Kolhapur	2024-01-15
Akash Verma	Delhi	2024-01-18
Neha Singh	Noida	2024-01-20
Kunal Mehta	Ahmedabad	2024-01-22
Riya Shah	Surat	2024-01-25
Sahil Khan	Bhopal	2024-01-27
Anjali Gupta	Indore	2024-02-01
Vikas Yadav	Lucknow	2024-02-03
Manish Jain	Jaipur	2024-02-05
Priya Malhotra	Chandigarh	2024-02-07
Deepak Rao	Hyderabad	2024-02-10
Swati Iyer	Chennai	2024-02-12
Arjun Nair	Kochi	2024-02-14
Nitin Pawar	Aurangabad	2024-02-16
Kavita More	Satara	2024-02-18
Yash Thakur	Raipur	2024-02-20

2. Videos with rating 8.0 and above

```
SELECT title, rating, type
FROM videos
WHERE rating >= 8.0;
```

OUTPUT:

Result Grid | Filter Rows:

title	rating	type
War Zone	8.4	Movie
Family Man	8.7	Series
Laugh Club	8.0	Series
Crime Files	8.2	Series
Mission X	8.1	Movie
Dark Truth	8.6	Series
Fighter Squad	8.3	Movie
Comedy Adda	8.1	Series
The Chase	8.0	Movie
Action Heroes	8.5	Movie
Hidden Secrets	8.4	Series

3. Total number of movies and series

```
SELECT type, COUNT(video_id) AS total_count
FROM videos
GROUP BY type;
```

OUTPUT:

Result Grid | Filter Rows:

type	total_count
Movie	11
Series	9

4. Subscription plans sorted by price (high to low)

```
SELECT plan_name, price, video_quality
FROM subscription_plans
ORDER BY price DESC;
```

OUTPUT:

Result Grid | Filter Rows:

plan_name	price	video_quality
Premium	799	4K
Standard	499	Full HD
Basic	199	HD
Mobile	149	SD

5. Total revenue generated till date

```
SELECT SUM(amount) AS total_revenue
FROM payments;
```

OUTPUT:

Result Grid | Filter Rows:

total_revenue
4590

6. Display subscription plans that support more than 2 devices

```
SELECT plan_name, max_devices, video_quality
FROM subscription_plans
WHERE max_devices > 2;
```

OUTPUT:

Result Grid | Filter Rows:

plan_name	max_devices	video_quality
Premium	4	4K

7. Top 6 highest rated videos

```
SELECT title, rating
FROM videos
ORDER BY rating DESC
LIMIT 6;
```

OUTPUT:

Result Grid | Filter Rows:

title	rating
Family Man	8.7
Dark Truth	8.6
Action Heroes	8.5
War Zone	8.4
Hidden Secrets	8.4
Fighter Squad	8.3

SUB-QUERIES

1. Users who have watched at least one video

```
SELECT user_id, full_name, city
FROM users
WHERE user_id IN (
   SELECT DISTINCT user_id
   FROM watch_history
);
```

OUTPUT:

Result Grid | Filter Rows:

user_id	full_name	city
1	Rahul Sharma	Mumbai
2	Amit Patil	Pune
3	Sneha Joshi	Nashik
4	Rohit Deshmukh	Nagpur
5	Pooja Kulkarni	Kolhapur
6	Akash Verma	Delhi
7	Neha Singh	Noida
8	Kunal Mehta	Ahmedabad
9	Riya Shah	Surat
10	Sahil Khan	Bhopal
11	Anjali Gupta	Indore
12	Vikas Yadav	Lucknow
13	Manish Jain	Jaipur
14	Priya Malhotra	Chandigarh
15	Deepak Rao	Hyderabad
16	Swati Iyer	Chennai
17	Arjun Nair	Kochi
18	Nitin Pawar	Aurangabad
19	Kavita More	Satara
20	Yash Thakur	Raipur

2. Videos watched by users from Mumbai (detailed output)

```
SELECT video_id, title, type, rating
FROM videos
WHERE video_id IN (
   SELECT video_id
   FROM watch_history
   WHERE user_id IN (
      SELECT user_id
      FROM users
      WHERE city = 'Mumbai'));
```

OUTPUT:

Result Grid | Filter Rows:

video_id	title	type	rating
2	Family Man	Series	8.7

3. Users whose subscription is currently ACTIVE

```
SELECT user_id, full_name , email
FROM users
WHERE user_id IN (
   SELECT user_id
   FROM user_subscriptions
   WHERE status = 'Active'
);
```

OUTPUT:

Result Grid | Filter Rows:

user_id	full_name	email
1	Rahul Sharma	rahul@gmail.com
2	Amit Patil	amit@gmail.com
4	Rohit Deshmukh	rohit@gmail.com
5	Pooja Kulkarni	pooja@gmail.com
7	Neha Singh	neha@gmail.com
8	Kunal Mehta	kunal@gmail.com
10	Sahil Khan	sahil@gmail.com
11	Anjali Gupta	anjali@gmail.com
12	Vikas Yadav	vikas@gmail.com
14	Priya Malhotra	priya@gmail.com
15	Deepak Rao	deepak@gmail.com
16	Swati Iyer	swati@gmail.com
18	Nitin Pawar	nitin@gmail.com
19	Kavita More	kavita@gmail.com
20	Yash Thakur	yash@gmail.com

4. Videos which have been watched at least once

```
SELECT video_id, title, release_year, rating
FROM videos
WHERE video_id IN (
   SELECT DISTINCT video_id
   FROM watch_history
);
```

OUTPUT:

Result Grid | Filter Rows:

video_id	title	release_year	rating
1	War Zone	2023	8.4
2	Family Man	2022	8.7
3	Laugh Club	2024	8.0
4	Love Forever	2021	7.6
5	Crime Files	2023	8.2
6	Mission X	2022	8.1
7	City Life	2020	7.4
8	Standup Nights	2021	7.9
9	Heart Beats	2022	7.8
10	Dark Truth	2024	8.6
11	Fighter Squad	2023	8.3
12	Broken Dreams	2021	7.5
13	Comedy Adda	2023	8.1
14	Forever Us	2024	7.7
15	The Chase	2022	8.0
16	Action Heroes	2024	8.5
17	Life Stories	2023	7.9
18	Fun Factory	2022	7.6
19	Love Again	2023	7.8
20	Hidden Secrets	2024	8.4

5. Users who paid more than average payment (with amount)

```
SELECT user_id, full_name, email
FROM users
WHERE user_id IN (
   SELECT user_id
   FROM payments
   WHERE amount > (
      SELECT AVG(amount)
      FROM payments
   )
);
```

OUTPUT:

Result Grid | Filter Rows:

user_id	full_name	email
1	Rahul Sharma	rahul@gmail.com
3	Sneha Joshi	sneha@gmail.com
5	Pooja Kulkarni	pooja@gmail.com
6	Akash Verma	akash@gmail.com
8	Kunal Mehta	kunal@gmail.com
10	Sahil Khan	sahil@gmail.com

JOINS

1. **Users with their active subscription plan**

```
SELECT u.full_name, sp.plan_name, us.status
FROM users u
JOIN user_subscriptions us ON u.user_id = us.user_id
JOIN subscription_plans sp ON us.plan_id = sp.plan_id
WHERE us.status = 'Active';
```

OUTPUT:

Result Grid | Filter Rows:

full_name	plan_name	status
Rahul Sharma	Standard	Active
Amit Patil	Basic	Active
Rohit Deshmukh	Mobile	Active
Pooja Kulkarni	Standard	Active
Neha Singh	Basic	Active
Kunal Mehta	Standard	Active
Sahil Khan	Premium	Active
Anjali Gupta	Basic	Active
Vikas Yadav	Standard	Active
Priya Malhotra	Premium	Active
Deepak Rao	Standard	Active
Swati Iyer	Basic	Active
Nitin Pawar	Mobile	Active
Kavita More	Basic	Active
Yash Thakur	Standard	Active

2. Watch history with user name and video title

```
SELECT u.full_name, v.title, w.watch_date, w.watch_time_min
FROM watch_history w
JOIN users u ON w.user_id = u.user_id
JOIN videos v ON w.video_id = v.video_id;
```

OUTPUT:

Result Grid | Filter Rows: | Export:

full_name	title	watch_date	watch_time_min
Rahul Sharma	Family Man	2024-02-01	40
Amit Patil	War Zone	2024-02-02	120
Sneha Joshi	Laugh Club	2024-02-03	25
Rohit Deshmukh	Love Forever	2024-02-04	90
Pooja Kulkarni	Crime Files	2024-02-05	50
Akash Verma	Mission X	2024-02-06	130
Neha Singh	City Life	2024-02-07	100
Kunal Mehta	Standup Nights	2024-02-08	30
Riya Shah	Heart Beats	2024-02-09	110
Sahil Khan	Dark Truth	2024-02-10	45
Anjali Gupta	Fighter Squad	2024-02-11	140
Vikas Yadav	Broken Dreams	2024-02-12	120
Manish Jain	Comedy Adda	2024-02-13	35
Priya Malhotra	Forever Us	2024-02-14	100
Deepak Rao	The Chase	2024-02-15	130
Swati Iyer	Action Heroes	2024-02-16	145
Arjun Nair	Life Stories	2024-02-17	55
Nitin Pawar	Fun Factory	2024-02-18	20
Kavita More	Love Again	2024-02-19	115
Yash Thakur	Hidden Secrets	2024-02-20	48

3. Genre-wise total watch time

```
SELECT g.genre_name, SUM(w.watch_time_min) AS total_watch_time
FROM watch_history w
JOIN videos v ON w.video_id = v.video_id
JOIN genres g ON v.genre_id = g.genre_id
GROUP BY g.genre_name;
```

OUTPUT:

Result Grid | Filter Rows:

genre_name	total_watch_time
Action	535
Drama	315
Comedy	110
Romance	415
Thriller	273

4. Total payment done by each user

```
SELECT u.full_name, SUM(p.amount) AS total_payment
FROM users u
JOIN payments p ON u.user_id = p.user_id
GROUP BY u.full_name;
```

OUTPUT:

Result Grid | Filter Rows:

full_name	total_payment
Rahul Sharma	499
Amit Patil	199
Sneha Joshi	799
Rohit Deshmukh	149
Pooja Kulkarni	499
Akash Verma	799
Neha Singh	199
Kunal Mehta	499
Riya Shah	149
Sahil Khan	799

5. Devices used by each user

```
SELECT u.full_name, d.device_type, d.last_login
FROM users u
JOIN devices d ON u.user_id = d.user_id;
```

OUTPUT:

Result Grid | Filter Rows:

full_name	device_type	last_login
Rahul Sharma	Mobile	2024-02-20
Amit Patil	Smart TV	2024-02-20
Sneha Joshi	Laptop	2024-02-19
Rohit Deshmukh	Tablet	2024-02-18
Pooja Kulkarni	Mobile	2024-02-20
Akash Verma	Smart TV	2024-02-17
Neha Singh	Laptop	2024-02-16
Kunal Mehta	Mobile	2024-02-15
Riya Shah	Tablet	2024-02-14
Sahil Khan	Smart TV	2024-02-13

CONCLUSION

The Video Streaming Management System Database project has successfully designed and implemented a comprehensive database system that efficiently manages and stores information related to video streaming operations. The system achieves its objectives by organizing user data, video content, subscriptions, watch history, devices, and payment records in a structured and reliable manner.

The database enhances data accuracy, consistency, and accessibility through proper table design and the effective use of primary and foreign key relationships. Its robust and scalable architecture ensures that the system can support future growth, including an increasing number of users, videos, and subscription plans.

This project demonstrates the practical application of SQL concepts, including database creation, data manipulation, and advanced querying using joins and subqueries. Overall, the Video Streaming Management System provides a strong foundation for managing real-world OTT platform data and showcases effective database design and analysis skills.